List of Experiments (Exception & Debugging)
EWA(UCS553)
(Week 5)
Jasleen Kaur 1024170091

1. Create a program that demonstrates the hierarchy of common exceptions such as ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, and NumberFormatException.

Code:

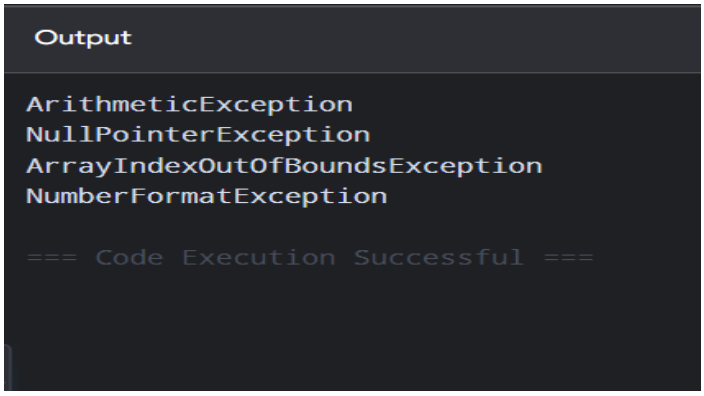
```
public class Main{
   public static void main(String[] args){
      try{
         int a=10/0;
      }catch(ArithmeticException e){
         System.out.println("ArithmeticException");
      }

      try{
         String s=null;
         System.out.println(s.length());
      }catch(NullPointerException e){
         System.out.println("NullPointerException");
      }

      try{
         int[] arr={1,2,3};
         System.out.println(arr[5]);
      }catch(ArrayIndexOutOfBoundsException e){
         System.out.println("ArrayIndexOutOfBoundsException");
      }

      try{
         int n=Integer.parseInt("abc");
      }catch(NumberFormatException e){
         System.out.println("NumberFormatException");
      }
   }
}
```

Output:

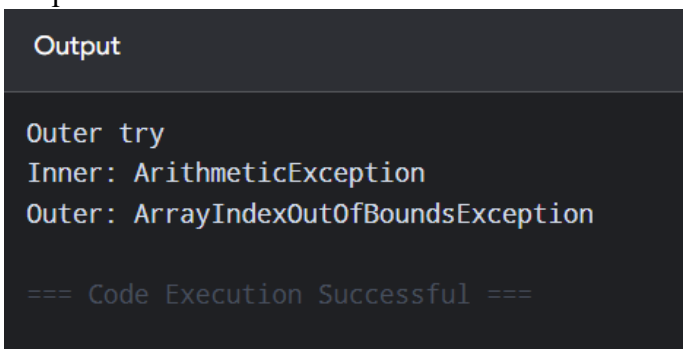


2. Write a program containing nested try-catch blocks. Generate different exceptions in the inner and outer blocks and demonstrate how exception propagation works.

Code:

```
public class Main{
   public static void main(String[] args){
      try{
         System.out.println("Outer try");
         try{
            int a=10/0;
         }catch(ArithmeticException e){
            System.out.println("Inner: ArithmeticException");
         }
         int[] arr={1,2,3};
         System.out.println(arr[5]);
      }catch(ArrayIndexOutOfBoundsException e){
         System.out.println("Outer: ArrayIndexOutOfBoundsException");
      }
   }
}
```

Output:



3. Develop a calculator that accepts two numbers and an operator (+, -, *, /). It should be able to handle the following exceptions: Division by zero, Invalid numeric input, Invalid operators. Use separate exception handlers for different errors.

Code:

```
import java.util.*;

public class Main{
```

```
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);

        try{
            System.out.print("Enter first number: ");
            double a=Double.parseDouble(sc.nextLine());

            System.out.print("Enter second number: ");
            double b=Double.parseDouble(sc.nextLine());

            System.out.print("Enter operator: ");
            char op=sc.nextLine().charAt(0);

            if(op!='+'&&op!='-'&&op!='*'&&op!='/')
                throw new IllegalArgumentException();

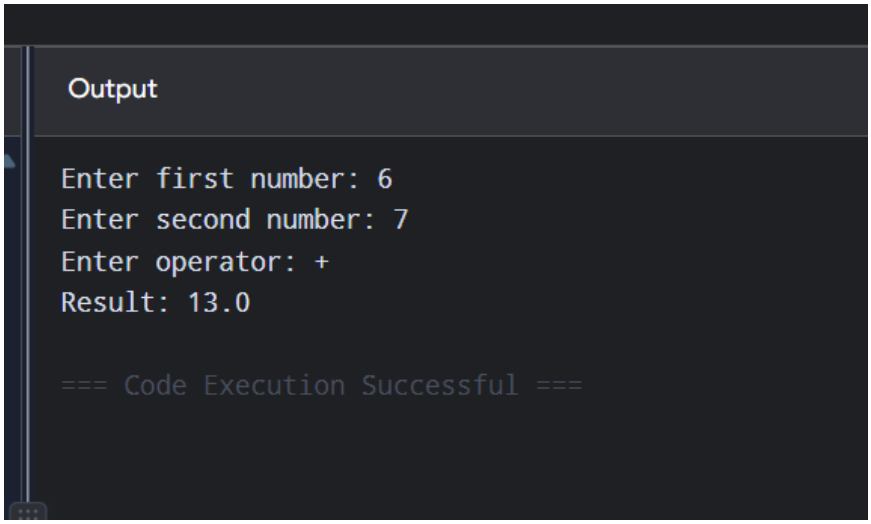
            if(op=='/'&&b==0)
                throw new ArithmeticException();

            double result;

            switch(op){
                case '+':result=a+b;break;
                case '-':result=a-b;break;
                case '*':result=a*b;break;
                default:result=a/b;
            }

            System.out.println("Result: "+result);
        }catch(NumberFormatException e){
            System.out.println("Invalid numeric input");
        }catch(ArithmeticException e){
            System.out.println("Division by zero");
        }catch(IllegalArgumentException e){
            System.out.println("Invalid operator");
        }
    }
}
```

Output:



4. Create a student result-processing program that accepts marks for several subjects. Marks must be between 0 and 100. Create a custom InvalidMarksException. Throw the exception when invalid marks are entered. Calculate total, percentage, and grade only when all input is valid. Display a meaningful error message.

Code:

```
import java.util.*;

class InvalidMarksException extends Exception{
   InvalidMarksException(String message){
      super(message);
   }
}

public class Main{
   public static void main(String[] args){
      Scanner sc=new Scanner(System.in);
      int total=0;

      try{
         System.out.print("Enter number of subjects: ");
         int n=sc.nextInt();

         for(int i=1;i<=n;i++){
            System.out.print("Enter marks: ");
            int marks=sc.nextInt();

            if(marks<0||marks>100)
               throw new InvalidMarksException("Invalid marks");

            total+=marks;
         }
```

```
            double percentage=(double)total/n;
            char grade;

            if(percentage>=90) grade='A';
            else if(percentage>=80) grade='B';
            else if(percentage>=70) grade='C';
            else if(percentage>=60) grade='D';
            else grade='F';

            System.out.println("Total: "+total);
            System.out.println("Percentage: "+percentage);
            System.out.println("Grade: "+grade);
        }catch(InvalidMarksException e){
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
Output

Enter number of subjects: 4
Enter marks: 80
Enter marks: 80
Enter marks: 98
Enter marks: 98
Total: 356
Percentage: 89.0
Grade: B

=== Code Execution Successful ===
```

5. Implement a simple banking application supporting deposit and withdrawal. Create a custom InsufficientBalanceException. Create a custom InvalidAmountException. Prevent withdrawal when the balance is insufficient. Prevent zero or negative deposits/withdrawals. Demonstrate exception propagation from one method to another. Display the account balance after successful transactions. Add AccountNotFoundException for invalid account numbers.

Code:

```
class InsufficientBalanceException extends Exception{
    InsufficientBalanceException(String message){
        super(message);
    }
}
```

```
class InvalidAmountException extends Exception{
   InvalidAmountException(String message){
      super(message);
   }
}

class AccountNotFoundException extends Exception{
   AccountNotFoundException(String message){
      super(message);
   }
}

class Bank{
   double balance=5000;

   void checkAccount(int account)throws AccountNotFoundException{
      if(account!=1001)
         throw new AccountNotFoundException("Account not found");
   }

   void deposit(double amount)throws InvalidAmountException{
      if(amount<=0)
         throw new InvalidAmountException("Invalid deposit amount");
      balance+=amount;
   }

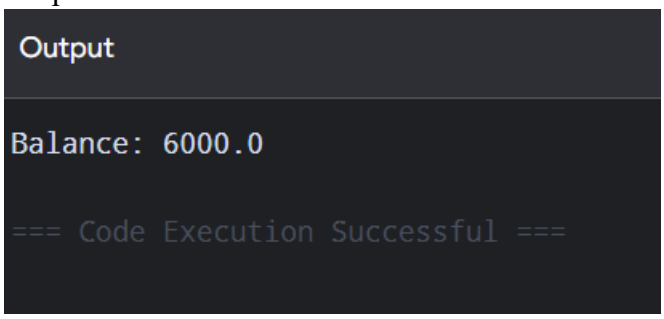
   void withdraw(double amount)throws
InvalidAmountException,InsufficientBalanceException{
      if(amount<=0)
         throw new InvalidAmountException("Invalid withdrawal amount");
      if(amount>balance)
         throw new InsufficientBalanceException("Insufficient balance");
      balance-=amount;
   }
}

public class Main{
   public static void main(String[] args){
      Bank b=new Bank();

      try{
         b.checkAccount(1001);
         b.deposit(2000);
         b.withdraw(1000);
         System.out.println("Balance: "+b.balance);
```

```
}catch(AccountNotFoundException|InvalidAmountException|InsufficientBalanceExcepti
on e){
            System.out.println(e.getMessage());
        }
    }
}
```

Output:



6. Develop a login system with exception-based validation. Create custom exceptions such as: InvalidUsernameException, InvalidPasswordException, AccountLockedException, Allow only a fixed number of failed login attempts. Lock the account after the maximum attempts. Use try-catch-finally appropriately. Display different messages for different failures.

Code:

```
class InvalidUsernameException extends Exception{
    InvalidUsernameException(String message){
        super(message);
    }
}

class InvalidPasswordException extends Exception{
    InvalidPasswordException(String message){
        super(message);
    }
}

class AccountLockedException extends Exception{
    AccountLockedException(String message){
        super(message);
    }
}

class Login{
    String username="admin";
    String password="1234";
    int attempts=0;
    boolean locked=false;
```

```
    void login(String u,String p)throws
      InvalidUsernameException,InvalidPasswordException,AccountLockedException{
        if(locked)
            throw new AccountLockedException("Account locked");

        if(!u.equals(username))
            throw new InvalidUsernameException("Invalid username");

        if(!p.equals(password)){
            attempts++;
            if(attempts>=3){
                locked=true;
                throw new AccountLockedException("Account locked");
            }
            throw new InvalidPasswordException("Invalid password");
        }

        System.out.println("Login successful");
    }
}

public class Main{
    public static void main(String[] args){
        Login l=new Login();

        try{
            l.login("admin","1111");
        }catch(InvalidUsernameException e){
            System.out.println(e.getMessage());
        }catch(InvalidPasswordException e){
            System.out.println(e.getMessage());
        }catch(AccountLockedException e){
            System.out.println(e.getMessage());
        }finally{
            System.out.println("Login process completed");
        }
    }
}
```

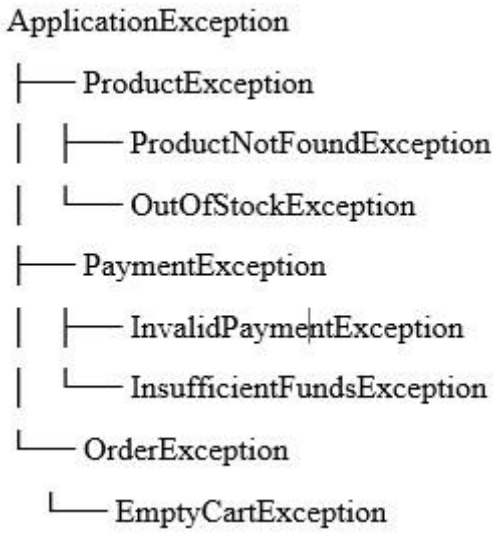
Output:

```
Output

Invalid password
Login process completed

=== Code Execution Successful ===
```

7. Design a small shopping/cart application that uses a complete exception hierarchy. Given below is the suggested hierarchy



Implement the appropriate operations like add/remove, payment, search products etc. Use appropriate exception handlers and debug the program using breakpoints and variable inspection.

Code:

```
class ApplicationException extends Exception{
   ApplicationException(String message){
      super(message);
   }
}

class ProductException extends ApplicationException{
   ProductException(String message){
      super(message);
   }
}

class ProductNotFoundException extends ProductException{
   ProductNotFoundException(String message){
      super(message);
```

```
    }
}

class OutOfStockException extends ProductException{
    OutOfStockException(String message){
        super(message);
    }
}

class PaymentException extends ApplicationException{
    PaymentException(String message){
        super(message);
    }
}

class InvalidPaymentException extends PaymentException{
    InvalidPaymentException(String message){
        super(message);
    }
}

class InsufficientFundsException extends PaymentException{
    InsufficientFundsException(String message){
        super(message);
    }
}

class OrderException extends ApplicationException{
    OrderException(String message){
        super(message);
    }
}

class EmptyCartException extends OrderException{
    EmptyCartException(String message){
        super(message);
    }
}

class Product{
    String name;
    double price;
    int stock;

    Product(String name,double price,int stock){
        this.name=name;
```

```
        this.price=price;
        this.stock=stock;
    }
}

class ShoppingCart{
    Product product;

    void search(String name)throws ProductNotFoundException{
        if(product==null||!product.name.equals(name))
            throw new ProductNotFoundException("Product not found");
        System.out.println("Product found: "+product.name);
    }

    void add(Product p)throws OutOfStockException{
        if(p.stock<=0)
            throw new OutOfStockException("Product out of stock");
        product=p;
        p.stock--;
        System.out.println("Product added: "+p.name);
    }

    void remove()throws EmptyCartException{
        if(product==null)
            throw new EmptyCartException("Cart is empty");
        System.out.println("Product removed: "+product.name);
        product=null;
    }

    void payment(double amount)throws
EmptyCartException,InvalidPaymentException,InsufficientFundsException{
        if(product==null)
            throw new EmptyCartException("Cart is empty");
        if(amount<=0)
            throw new InvalidPaymentException("Invalid payment");
        if(amount<product.price)
            throw new InsufficientFundsException("Insufficient funds");
        System.out.println("Payment successful");
    }
}

public class Main{
    public static void main(String[] args){
        ShoppingCart cart=new ShoppingCart();
        Product p=new Product("Laptop",50000,2);
```

```
        try{
            cart.search("Laptop");
        }catch(ProductNotFoundException e){
            System.out.println(e.getMessage());
        }

        try{
            cart.add(p);
        }catch(ProductException e){
            System.out.println(e.getMessage());
        }

        try{
            cart.payment(50000);
        }catch(PaymentException e){
            System.out.println(e.getMessage());
        }catch(OrderException e){
            System.out.println(e.getMessage());
        }

        try{
            cart.remove();
        }catch(OrderException e){
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

